Scrubgnome — Technical Whitepaper

A local, private video scrubber that strips location-revealing metadata and audio from videos. Version 1.0 · November 2026

Abstract

Videos can leak the location where they were recorded in two ways: hidden file metadata (GPS, timestamp, device) and location-revealing background sound. `Scrubgnome` is a modular, local-only tool that removes both. Its headline capability is defeating a specific and under-appreciated attack: acoustic Doppler geolocation, where the pitch of an aircraft flyover shifts as the plane moves (the Doppler effect) and that pitch-versus-time curve is matched against public flight-tracking (ADS-B) data plus terrain to place a recording to roughly 100 metres — sometimes from under a minute of audio. Scrubgnome strips the metadata and suppresses the sub-audible aircraft tones that make this possible, treats each defense as a swappable module, and ships with a one-click launcher, a browser UI, and optional Audacity helpers. It runs entirely on the user's machine; nothing is uploaded.

1. The attack

The geolocation technique proceeds roughly as follows:

1. Extract a spectrogram of the recording and isolate the sweeping tonal lines of any aircraft flyover (engine/fan tones whose pitch glides downward as the plane passes closest, then recedes).
2. Pull historical ADS-B transponder tracks for aircraft in the air near the recording's approximate time.
3. For each candidate recording location, model the expected Doppler curve — adjusting for terrain via a digital elevation model — and score it against the observed curve.
4. The best-scoring location is the likely origin.

The attack needs three things, and removing **any** of them degrades it:

- **(a)** a recoverable Doppler curve in the audio,
- **(b)** an approximate recording time (to pull the right flight data),
- **(c)** access to ADS-B history (outside the defender's control).

`Scrubgnome` attacks (a) and (b).

2. Threat model & design principles

- **Local only.** All processing is on-device; no uploads. The browser UI binds to `127.0.0.1`.
- **Modular.** Each defense is a replaceable stage so modules can be reused, upgraded, or swapped (a design preference carried throughout the project).
- **Honest about guarantees.** No audio scrubber can *prove* a curve is unrecoverable against every possible matcher. We measure and report relative strength; we do not claim certainty.
- **Deliberately scoped.** We built the *defensive* detector (find/flag/remove a flyover). We did **not** build a Doppler-curve *measurement/geolocation* simulator, as that is the offensive capability itself.

3. Pipeline overview



A separate detection path (Audacity plug-ins) can flag or duck flyovers in an editor. The UI exposes the pipeline as four preset "protection levels."

4. Module 1 — Metadata scrubber (`hmd/scrub.py`)

Removes the time/location/device anchors the attack uses for step (b):

- Strips container and per-stream tags: `creation_time`, ISO-6709 GPS (`location`), `make` / `model`, `encoder`, handler names, vendor IDs.
- Drops chapters and data/telemetry side-tracks (which can carry GPS).
- Audio and video are **stream-copied** — no re-encode, no quality loss.

- Optional decoy `creation_time` can be written to mislead a time-gated attacker.
- A 0–100 **leak score** (location 50 / time 35 / device 15) quantifies exposure before and after.

5. Module 2 — Perceptual cleaner (`hmd/clean.py`)

The core audio defense. It erases the faint tonal evidence while leaving audible content intact, using psychoacoustic masking.

5.1 Masking model

Per STFT frame (2048-pt, 75 % overlap): - The spectrum is grouped into ~25 Bark critical bands. - A masking threshold per band is estimated via the Schroeder spreading function plus an offset; bins below threshold are, by definition, **inaudible**. - Bins below the absolute threshold of hearing (Terhardt ATH, referenced to an assumed playback SPL) are zeroed.

5.2 Substituting the inaudible content

Inaudible bins carry the Doppler evidence. We replace their **magnitude** with the band's noise-floor level — flattening any faint tonal ridge — and re-phase them. Phase handling is the subtle part, and we expose it as `phase_mode`:

- **`noise` (default).** Phase is taken from a single *continuous* noise signal's STFT. Because that phase is temporally coherent (it comes from one real signal), overlap-add reconstructs **smooth broadband comfort noise** — no "musical noise." Yet it still *decorrelates* any steady tone, so the tone averages away in a long-integration spectrogram. Best overall balance.
- **`random`.** Independent random phase per bin per frame. Also decorrelates, but the per-frame jumps produce musical noise that sounds like wind/pops. (Legacy.)
- **`preserve`.** Keep original phase. Cleanest possible sound, but a reduced-magnitude steady tone stays *coherent* and can still integrate into a spectrogram ridge — so this **weakens** privacy. Offered for comparison and for the "cleanest sound" level.

The substitution level is **temporally smoothed** over several frames (`smooth_frames`, default 7) so the noise bed does not flutter.

5.3 Why `preserve` leaks (measured)

On a test clip with a faint 520 Hz tone 24 dB above the local floor:

phase_mode	tone above floor (after)	low-band frame flutter
original	24.0 dB	2.69
random	8.0 dB	2.09
noise	**10.1 dB**	**1.77** (smoothest)
preserve	23.8 dB (≈ unflattened)	1.59

`noise` flattens the tone nearly as well as `random` while being *smoother than the original recording* — hence it is the default.

6. De-tinny post-processing (`hmd/post.py`)

Masked-bin substitution tends to thin the low-mids (voice "body") and brighten the top, which sounds tinny. `post.warmth()`: - Measures the original's long-term 1/3-octave magnitude envelope and re-tilts the cleaned audio to match it (restores tonal balance), clamped to avoid boosting near-silent bands into roars. - Applies a gentle high-shelf to tame papery hiss. - Operates on the **broad** spectral envelope only, so it does **not** reintroduce the narrow, moving Doppler tone.

7. Protection levels (UI presets)

Level	Name	Pipeline / parameters	Intent
1	Metadata only	scrub only; audio untouched	No jet present; keep audio pristine
2	Standard	scrub + clean(offset 5 dB, SPL 90, phase=noise, smooth 7, warmth 1)	Everyday strong privacy
3	Cleanest sound	scrub + clean(offset 8 dB, SPL 95, phase=preserve, warmth 1)	Best fidelity, ~partial privacy by design
4	Max privacy	scrub + clean(offset 2 dB, SPL 80, phase=noise, smooth 9, warmth 1)	Privacy over fidelity

Level 4's "100 %" is a *goal*, not a measured guarantee.

8. Flyover detection — Audacity plug-ins (`audacity/`)

Two Nyquist plug-ins, defensive only (no curve measurement):

- **Label Jet Flyovers** (analyze) — adds a label per detected pass.
- **Jet Flyover Suppressor** (effect) — ducks the audio during each pass via a slew-limited gain envelope (`snd-from-array`).

Detection treats a flyover as a **swell**: low-band (sub-split) energy, in 100 ms blocks, rising a set number of dB **above the clip's median low-band level** and sustained for a minimum duration. Using the median as a baseline means steady low tones (a bass note, a droning fan) raise the baseline too and do not trip it — only departures do. Defaults (4 dB above baseline, 2 s) were tuned on synthetic flyovers; they are a starting point, not a guarantee, and are **untested inside Audacity** on this build.

9. Tooling

- **`setup_check.py`** — verifies Python 3.9+, numpy, scipy, ffmpeg/ffprobe (+AAC encoder); offers installs across winget/choco/scoop, brew, apt, dnf (EPEL + RPM Fusion + CRB on RHEL/Rocky), pacman.
- **`prepare_windows.ps1` + `Launch Scrubgnome.cmd`** — a no-Python-required Windows launcher. It self-elevates (one UAC prompt), checks/installs Python (python.org silent, all-users, on PATH), numpy/scipy (pip), and ffmpeg (gyan.dev build into `Program Files\ffmpeg\bin` + system PATH), streams progress to a dark status-bar window on a background worker thread, then **launches the app automatically**. `prepare_linux.sh` is the CLI equivalent (distro packages, venv fallback).
- **`ui.py`** — a stdlib-only local web server (`127.0.0.1:8765`) with the four-level UI, metadata leak table, WAV export for Audacity, in-page player, and Twig-voiced tooltips with a plain-language/geek-mode toggle.

10. Limitations & future work

- **Not a guarantee.** The cleaner reduces, and in testing largely erases, tonal evidence, but against an unknown matcher no certainty is claimed.
- **Memory-bound.** The cleaner holds the full audio track in memory.
- **Audacity plug-ins untested in-app** on this build (algorithm validated in Python; Nyquist is paren-checked).
- **Windows launcher** has not yet been run on a clean Windows box end-to-end.
- **Roadmap modules** (designed, not all built): ML source separation of the flyover; a random time-varying pitch/time **curve warper** that corrupts the curve while leaving speech intact; **decoy** synthetic Doppler sweeps; and a red-team **attack simulator** to measure location error before/after. The attack-simulator / curve-measurement component was intentionally **not** built, as it is the offensive geolocation capability itself.

Scrubgnome is a personal privacy project. It processes everything locally and sends nothing anywhere.
